

Chapter 8: Sorting in linear time

1) Comparison sort algorithms: The sorted order determined by these algorithms depends only upon comparisons between input elements. For example: merge sort, heap sort, quick sort.

→ In order to sort n elements, any comparison sort must make $\Omega(n \log n)$ comparisons in the worst case. Thus, merge sort and heap sort are asymptotically optimal, and no comparison sort exists that is faster by more than a constant factor.

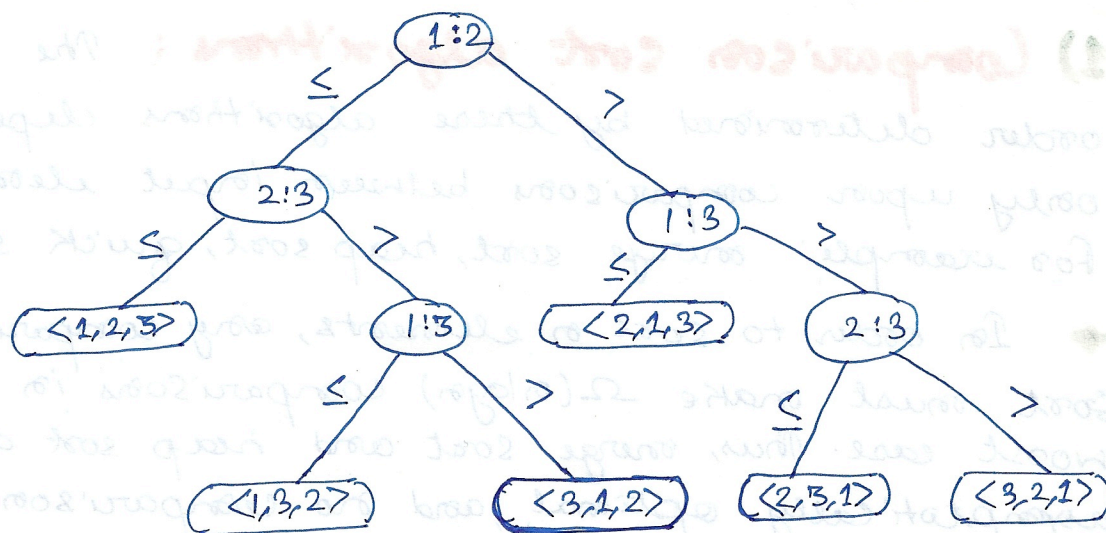
→ Given two elements a_i and a_j , we can only perform one of the tests: $a_i < a_j$, $a_i \leq a_j$, $a_i = a_j$, $a_i \geq a_j$ or $a_i > a_j$, to determine their relative order. Without loss of generality, we can assume that all comparisons we make in our comparison sorts have the form $a_i \leq a_j$.

2) The decision-tree model of comparison

sorts: Comparison sorts can be viewed abstractly in terms of decision trees. A decision tree is a full binary tree that represents the comparisons between the elements that are performed by a particular sorting algorithm operating on an input of a given size. [A full binary tree is a binary tree where every node is either a leaf or has a degree of exactly 2].

For example, we have the following decision tree corresponding to insertion sort algorithm operating on

an input sequence of three elements:-



→ An internal node annotated by $i:j$ indicates a comparison between elements a_i and a_j . A leaf annotated by the permutation $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ indicates the ordering $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$.

→ A correct sorting algorithm must be able to produce each permutation of its input. Hence, each of the $n!$ permutations on n elements must appear as one of the leaves of the decision tree, and that each of these leaves must be reachable from the root.

3) A lower bound for the worst case:-

The worst-case number of comparisons performed by a sorting algorithm is represented by the length of the longest path from the root of a decision tree to any of its reachable leaves. In other words, the worst case number of comparisons for a comparison sort equals the height of its decision tree.

Therefore, a lower bound on the running time of any comparison sort would equal a lower bound on the heights of all decision trees in which each permutation appears as a reachable leaf.

Theorem: Any comparison sort algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof: Consider a decision tree of height h with l reachable leaves corresponding to a comparison sort on n elements. Because each of the $n!$ permutations of the input appears as a leaf, we have $n! \leq l$. Since, a binary tree of height h has a maximum of 2^h leaves, we have :-

$$n! \leq l \leq 2^h$$

Taking logarithm we have :-

$$h \geq \lg(n!) \quad [\text{Since } \lg \text{ function is monotonically increasing}]$$
$$= \Omega(n \lg n)$$

Corollary: Heapsort and mergesort are asymptotically optimal comparison sorts.

Proof: This is true because the $O(n \lg n)$ upper bounds on the running times for heapsort and mergesort match the $\Omega(n \lg n)$ worst-case bound.

4) Counting sort :-

Counting sort assumes that each of the n input elements is an integer in the range 0 to K , for some integer K .

The basic idea of counting sort is to determine, for each input element x , the number of elements less than x . This information can be used to place element x directly into its appropriate position in the output array. For example, if there are 17 elements less than x , then x belongs to O/P position 18.

This scheme must be modified slightly to handle the situation in which several elements have the same value, since we cannot put them all in the same position.

→ In the following pseudocode, we assume that the input is in array $A[1..n]$ and thus $\text{length}[A] = n$. The O/P is stored in array $B[1..n]$ and the array $C[0..K]$ provides temporary storage.

→ The array C can be called a counting array, where the element at index i holds a count of no. of elements in A less than or equal to i . Thus, the location of element i of I/P array A is index $C[i]$ in O/P array. If there are more elements in A having a value equal to i , they are stored at locations $C[i]-1, C[i]-2$ so forth.

COUNTING_SORT (A, B, K)

```
1 for i ← 0 to K
2   do c[i] ← 0
3 for j ← 1 to length[A]
4   do c[A[j]] ← c[A[j]] + 1
5 ▷ every c[i] now contains the no. of elements in A
   which are equal to i
6 for i ← 1 to K
7   do c[i] ← c[i] + c[i-1]
8 ▷ every c[i] now contains the no. of elements less than
   or equal to i
9 for j ← length[A] down to 1
10  do B[c[A[j]]] ← A[j]
11  c[A[j]] ← c[A[j]] - 1
```

Run time analysis: We have 2 for loops that take time $\Theta(K)$ and 2 for loops that take time $\Theta(n)$. Thus, the overall time is $\Theta(K+n)$. Usually, we use counting sort when $K = O(n)$, in which case the running time of counting sort is $\Theta(n)$.

→ Counting sort beats the lower bound of $\Omega(n \lg n)$ because it is not a comparison sort. No comparisons between 1/P elements occur anywhere in the code. Instead counting sort uses the values of the elements to index into the counting array.

→ An important property of counting sort is that it is stable: numbers with the same value, appear in the O/P array in the same order as they do in the I/P array. That is, ties between 2 numbers are broken by the rule that whichever number appears first in the I/P array appears first in the O/P array as well.

This property is important usually when satellite data are carried around with the element being sorted.

5) Radix sort: Sometimes the key of a record may be a combination of several fields. In such cases, comparing 2 keys directly is not possible. You can directly compare 2 fields in the same position in the 2 keys. This means that we need to sort the records on each of the fields of the keys one after the other.

The order in which the fields are picked for sorting is determined by their relative importance. Since the field which is picked in the end has the final and hence the largest impact on the sorted order, the most significant field should be the last one to go. On the other hand, the ordering imposed by the first-to-go field is masked by that imposed by the following fields. Hence, the least important field should be the first one to be picked up.

Thus, the technique of radix sort is counter to intuition, which would pick the most significant digit first.

→ To sort a sequence of n numbers having d digits each, You can divide each number into d bins such the one digit goes to each bin. Then you sort the numbers based upon the digits in a bin, starting from the least significant bin. Thus, only d passes through a sequence of n numbers is required.

For eg:-

329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839
7 3-digit no.s	Sorted on 1st bin from left to right	sorted on 2nd bin from right	sorted on most significant bin

→ Radix sort uses as a subroutine some other sorting algorithm that is stable. Stability of that algorithm is crucial for correctness of radix sort. For example, consider the 4 dates:-

1/1/07	1/2/07	1/1/07	13/2/03
1/5/07	1/5/07	13/2/03	1/1/07
2/7/06	2/7/06	1/5/07	1/5/07
13/2/03	13/2/03	2/7/06	2/7/06
D-M-Y	Sorted on date	Sorted on month	Sorted on year

When you are sorting the dates on year, you find 2 dates having an year value of 07: 1/1/07 & 1/5/07. You should not ~~swap them~~ change their relative position in the O/P array, otherwise the resulting sequence would not be correctly sorted. Hence, stability is important.

→ The following procedure takes as input an n -element array A having d digits, where digit 1 is the least significant and digit d is the most significant.

RADIX-SORT(A, d)

```
1 for  $i \leftarrow 1$  to  $d$ 
2   do use a stable sort to sort array  $A$  on digit  $i$ 
```

→ **Lemma:** Given n d -digit numbers in which each digit can take one out of K possible values, RADIX-SORT correctly sorts these numbers in $\Theta(d(n+K))$ time.

Proof: The analysis of running time of RADIX-SORT depends upon the running time of the stable sort used as an intermediate sorting algorithm. If each digit is in the range 0 to $K-1$ (i.e. takes 1 out of K possible values), counting sort is the obvious choice. Since, each pass of counting sort over n numbers takes $\Theta(n+K)$ and there are d passes, the running time of radix sort is $\Theta(d(n+K))$.

If $K = O(n)$ and d is a constant, radix sort runs in linear time.

→ **Lemma:** Given n b -bit numbers, where each number can be divided into $\frac{b}{r}$ r -bit digits ($r \leq b$), radix sort correctly sorts these numbers in $\Theta\left(\frac{b}{r}(n+2^r)\right)$ time.

Proof: Since each digit has r bits, each digit is an integer in the range 0 to 2^r-1 . Hence, value of k in counting sort = 2^r-1 . Each pass of counting sort takes $\Theta(n+2^r)$ and there are $\frac{b}{r}$ passes. So, running time = $\Theta\left(\frac{b}{r}(n+2^r)\right)$.

→ If we have some flexibility in how to break each key into digits, what should be the chosen value of r (i.e. no. of bits in a digit)? We need to choose the value of r that minimizes the expression $\left(\frac{b}{r}\right)(n+2^r)$. We have 2 cases:-

(i) $b < \lfloor \lg n \rfloor$: (say $b=2$ and $n=8, \lg 8 = 3 > 2$)

Choosing $r=b$ yields a running time of $\left(\frac{b}{b}\right)(n+2^b) = \Theta(n)$ which is asymptotically optimal. Here there is a single digit.

(ii) $b \geq \lfloor \lg n \rfloor$:

Choosing $r = \lfloor \lg n \rfloor$ gives the best time to within a constant factor. running time =

$$\Theta\left(\frac{b}{\lg n} \times \Theta(n)\right) = \Theta\left(\frac{bn}{\lg n}\right). \text{ If we increase } r \text{ above } \lfloor \lg n \rfloor$$

the 2^r term in the numerator increases faster than the r term in the denominator. And so, increasing r above $\lfloor \lg n \rfloor$ yields a running time of $\Omega\left(\frac{bn}{\lg n}\right)$.

If we were to decrease α below $\lfloor \lg n \rfloor$, then the
 b/α term increases and the $n+2^n$ term remains at
 $\Theta(n)$.

$$\Theta\left(\frac{n}{\alpha}\right)(n+2^n) \text{ time.}$$

Proof: Since each digit has $2^{b/\alpha}$ possible values, and there are n digits, the total number of possible values is $2^{n \cdot b/\alpha}$. For each possible value, we need to check if it is a power of 2. This takes $\Theta(\log n)$ time. So, the total time is $\Theta\left(\frac{n}{\alpha}\right)(n+2^n)$.

If we have some flexibility in how to break each
 key into digits, what should be the chosen value of α ?
 (i.e. one of bits is a digit? the goal is to choose the
 value of α that minimizes the expression
 $\frac{n}{\alpha}(n+2^n)$. We have 2 cases:

$$\textcircled{1} \alpha < \lg n : \text{ (say } \alpha = 2 \text{ and } n = 8, 8 \cdot 2 = 16 > 2)$$

Choosing $\alpha = 2$ yields a running time of

$$\frac{n}{2}(n+2^n) = \Theta(n) \text{ which is asymptotically optimal.}$$

Here there is a single digit.

$$\textcircled{2} \alpha \geq \lg n :$$

Choosing $\alpha = \lfloor \lg n \rfloor$ gives the best time to

minimize a constant factor. Running time:

$$\Theta\left(\frac{n}{\lg n}\right) \times \Theta(n) = \Theta\left(\frac{n^2}{\lg n}\right). \text{ If we choose } \alpha \text{ above } \lg n$$

the 2^n term in the numerator becomes faster than the
 n term in the denominator. For $n = 20$, increasing α
 above $\lfloor \lg n \rfloor$ yields a running time of $\Omega\left(\frac{n^2}{\lg n}\right)$.